

BUILDING HEALTHCARE STROKE DETECTION BY MACHINE LEARNING



Prepared for: Prof. ROBERTA SICILIANO

Prepared by: USWA UROOJ

Contents

OBJECTIVE	2
IMPORTING LIBRARIES & CONFIGURATION.....	2
EXPLANATION	3
DATA ANALYSIS	4
EXPLANATION	4
DATA INFORMATION	5
EXPLANATION	6
MISSING VALUES HANDLING	6
EXPLANATION	7
UNIQUE VALUES.....	14
EXPLANATION	15
NORMALIZATION	16
EXPLANATION	16
INITIALIZATION.....	16
EXPLANATION	17
LOGISTIC REGRESSION	18
CONCLUSION.....	20

OBJECTIVE

This project aims to develop and implement an advanced system that utilizes machine learning algorithms to detect and prevent strokes in healthcare settings. By leveraging predictive models and anomaly detection techniques, the project aims to enhance the early identification and intervention of stroke risks, thereby minimizing the incidence and severity of strokes and improving patient outcomes. Through continuous learning and adaptation, the system will evolve to effectively anticipate and address emerging risk factors associated with strokes, ultimately safeguarding the well-being of patients and optimizing healthcare resources.

IMPORTING LIBRARIES & CONFIGURATION

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
from datetime import datetime

##### Sklearn #####
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
pd.options.mode.copy_on_write = True # Allow re-write on variable
sns.set_style('darkgrid') # Seaborn style
warnings.filterwarnings('ignore') # Ignore warnings
pd.set_option('display.max_columns', None) # Setting this option will print all columns of a dataframe
pd.set_option('display.max_colwidth', None) # Setting this option will print all of the data in a feature
```

EXPLANATION

Importing Libraries

- pandas: Data manipulation and analysis library.
- numpy: Numerical computing library.
- seaborn: Statistical data visualization library based on matplotlib.
- matplotlib.pyplot: Plotting library for creating visualizations.
- warnings: Python's built-in module for handling warnings.
- datetime: Module to work with dates and times.

Importing Specific Functions and Classes from Sklearn

- MinMaxScaler: Used for scaling features to a range.
- train_test_split: For splitting dataset into training and testing subsets.
- GridSearchCV: Performs exhaustive search over specified parameter values for an estimator.
- RandomForestClassifier: Ensemble learning method for classification.
- metrics: Module containing various evaluation metrics.
- LogisticRegression: Classifier implementing logistic regression.
- SVC: Support Vector Classification.
- DecisionTreeClassifier: Classification algorithm based on decision trees.
- KNeighborsClassifier: Classifier based on k-nearest neighbors.

Setting Pandas and Seaborn Options

- pd.options.mode.copy_on_write = True: Allows re-writing on variables, improves performance by reducing memory usage.
- sns.set_style('darkgrid'): Sets the aesthetic style of the plots to 'darkgrid' in Seaborn.
- warnings.filterwarnings('ignore'): Ignores warnings generated during code execution.

- `pd.set_option('display.max_columns', None)`: Displays all columns of a DataFrame without truncation.
- `pd.set_option('display.max_colwidth', None)`: Displays all data in a feature without truncation.

DATA ANALYSIS

```
data = pd.read_csv('./healthcare-dataset-stroke-data.csv')
data.head()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

EXPLANATION

`import pandas as pd`: This line imports the Pandas library and aliases it as 'pd'. Pandas is a powerful library for data manipulation and analysis.

- `data = pd.read_csv('./creditcard_2023.csv')`: This line reads the contents of a CSV file named 'creditcard_2023.csv' located in the current directory (indicated by './') and assigns it to the variable 'data'. The data is read into a Pandas DataFrame, which is a two-dimensional, tabular data structure.
- `data.head()`: This line prints the first few rows of the dataset using the `head()` method. By default, it displays the first 5 rows. This is a useful way to quickly inspect the structure and content of the loaded data.

DATA INFORMATION

```
# Drop column = 'id'
data.drop(columns='id', inplace=True)
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 5110 non-null  object
1   age                   5110 non-null  float64
2   hypertension           5110 non-null  int64
3   heart_disease          5110 non-null  int64
4   ever_married           5110 non-null  object
5   work_type              5110 non-null  object
6   Residence_type         5110 non-null  object
7   avg_glucose_level      5110 non-null  float64
8   bmi                   4909 non-null  float64
9   smoking_status         5110 non-null  object
10  stroke                 5110 non-null  int64
dtypes: float64(3), int64(3), object(5)
memory usage: 439.3+ KB
```

```
round(data.describe(include='all'), 2)
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
count	5110	5110.00	5110.0	5110.00	5110	5110	5110	5110.00	4909.00	5110	5110.00
unique	3	NaN	NaN	NaN	2	5	2	NaN	NaN	4	NaN
top	Female	NaN	NaN	NaN	Yes	Private	Urban	NaN	NaN	never smoked	NaN
freq	2994	NaN	NaN	NaN	3353	2925	2596	NaN	NaN	1892	NaN
mean	NaN	43.23	0.1	0.05	NaN	NaN	NaN	106.15	28.89	NaN	0.05
std	NaN	22.61	0.3	0.23	NaN	NaN	NaN	45.28	7.85	NaN	0.22
min	NaN	0.08	0.0	0.00	NaN	NaN	NaN	55.12	10.30	NaN	0.00
25%	NaN	25.00	0.0	0.00	NaN	NaN	NaN	77.24	23.50	NaN	0.00
50%	NaN	45.00	0.0	0.00	NaN	NaN	NaN	91.88	28.10	NaN	0.00
75%	NaN	61.00	0.0	0.00	NaN	NaN	NaN	114.09	33.10	NaN	0.00
max	NaN	82.00	1.0	1.00	NaN	NaN	NaN	271.74	97.60	NaN	1.00

EXPLANATION

- We drop id columns, because its a unique identifier number
- We have 5110 samples, with no null values

MISSING VALUES HANDLING

```
data.isna().sum()
```

```
gender          0
age             0
hypertension     0
heart_disease    0
ever_married     0
work_type        0
Residence_type   0
avg_glucose_level 0
bmi             201
smoking_status   0
stroke           0
dtype: int64
```

```
print((data.isna().sum()/len(data))*100)
```

```
gender          0.000000
age             0.000000
hypertension     0.000000
heart_disease    0.000000
ever_married     0.000000
work_type        0.000000
Residence_type   0.000000
avg_glucose_level 0.000000
bmi             3.933464
smoking_status   0.000000
stroke           0.000000
dtype: float64
```

```
### Missing values in BMI columns is about 4% , we drop them.
data.dropna(how='any', inplace=True)
```

```
cols = data.columns[:-1]
cols
```

```
Index(['gender', 'age', 'hypertension', 'heart_disease', 'ever_married',
       'work_type', 'Residence_type', 'avg_glucose_level', 'bmi',
       'smoking_status'],
      dtype='object')
```

data

data

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
2	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
5	Male	81.0	0	0	Yes	Private	Urban	186.21	29.0	formerly smoked	1
...	...	...	...	...	...	...	...	...	...	...	...
5104	Female	13.0	0	0	No	children	Rural	103.08	18.6	Unknown	0
5106	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

4909 rows × 11 columns

EXPLANATION

- There is 201 samples with no values in bmi column , its about 4% of data. For better result we drop them
- `data.isna().sum()`: Calculates the number of missing values (NaN) in each column of the DataFrame data.
- `print((data.isna().sum()/len(data))*100)`: Prints the percentage of missing values for each column in the DataFrame data.
- `data.dropna(how='any', inplace=True)`: Drops rows containing any missing values (NaN) from the DataFrame data. The `how='any'` parameter specifies to drop rows if any NaN value is present in the row. The `inplace=True` parameter ensures that the changes are

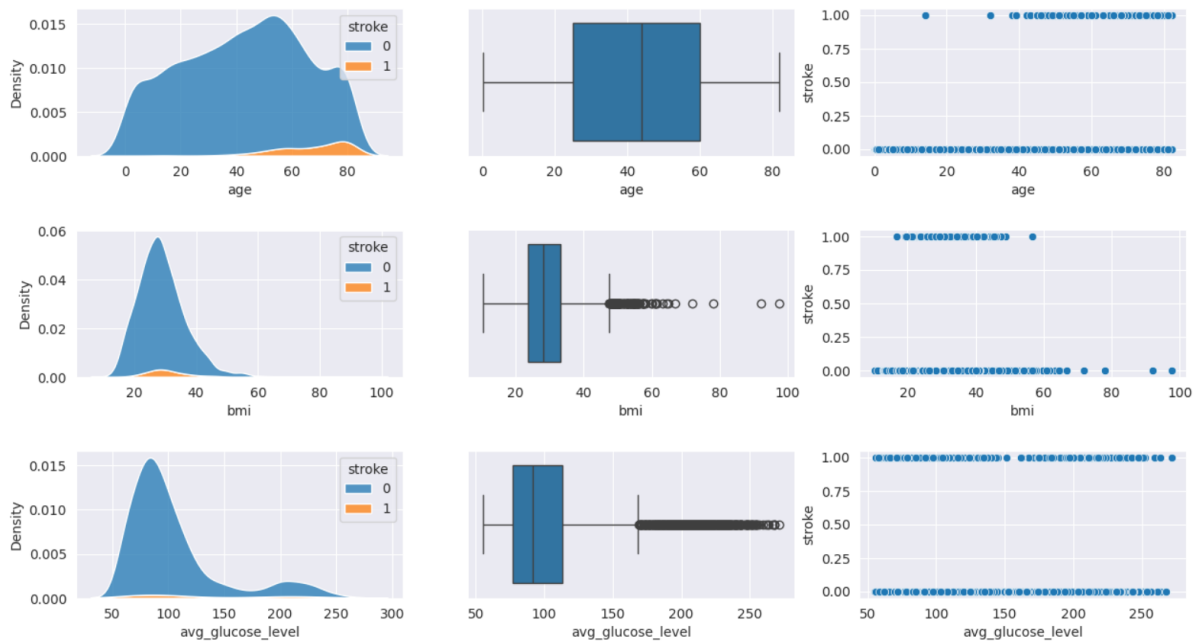
made directly to the original DataFrame data.

- cols = data.columns[:-1]: Retrieves all columns except the last one (assuming the last column is the target variable). This is stored in the variable cols.
- data: Displays the DataFrame after dropping rows with missing values.

EXPLORATORY DATA ANALYSIS

```
numeric_columns = ['age', 'bmi', 'avg_glucose_level']  
categorical_columns = ['gender', 'hypertension', 'heart_disease', 'ever_married', 'work_type', 'Residence_type', 'smoking_status', 'stroke']
```

```
i = 0  
fig, ax = plt.subplots(3, 3, figsize=(15, 8))  
plt.subplots_adjust(hspace = 0.5)  
for num_col in numeric_columns :  
    sns.kdeplot(x=num_col, hue='stroke', data=data, multiple='stack', ax=ax[i,0])  
    sns.boxplot(x=num_col, data=data, ax=ax[i, 1])  
    sns.scatterplot(x=num_col, y='stroke', data=data, ax=ax[i, 2])  
    i+=1  
plt.show()
```



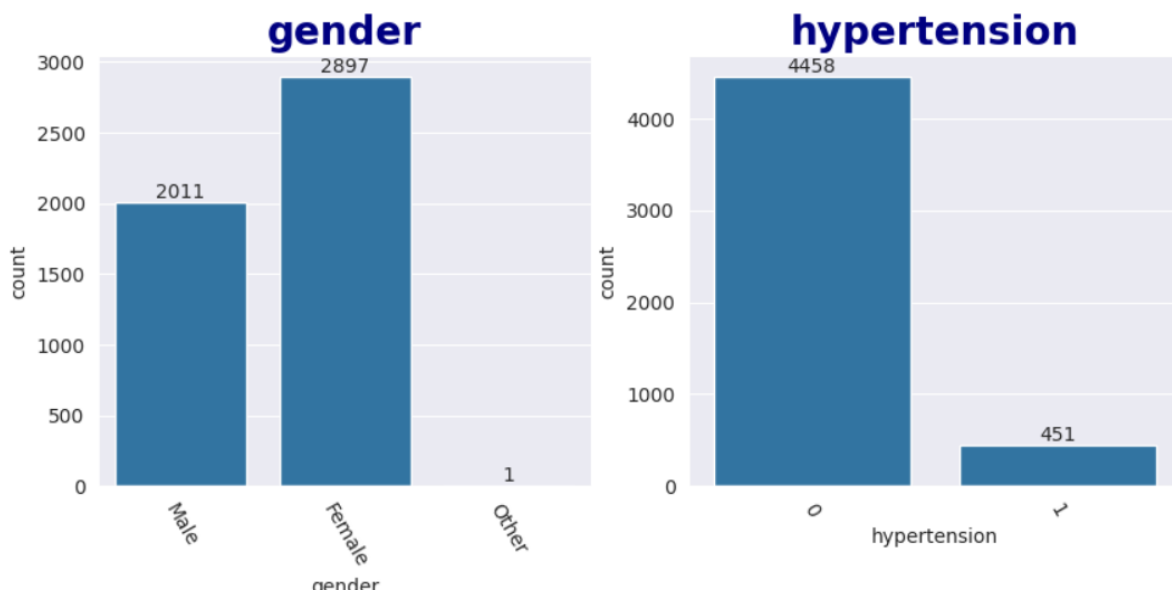
```

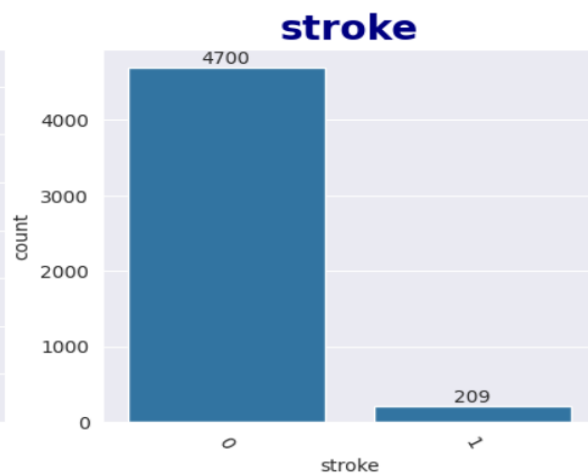
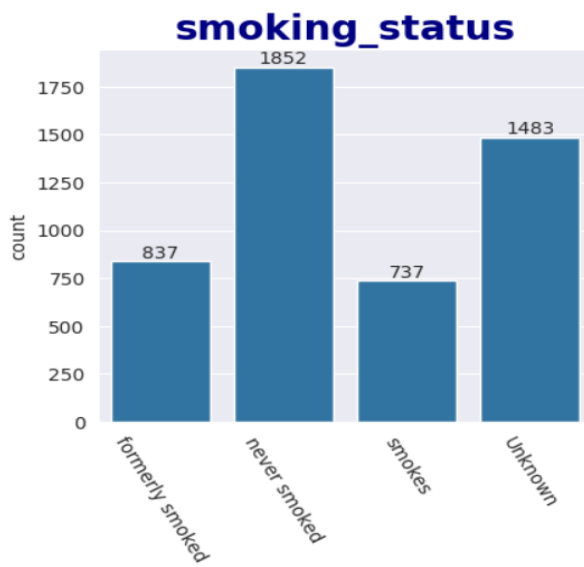
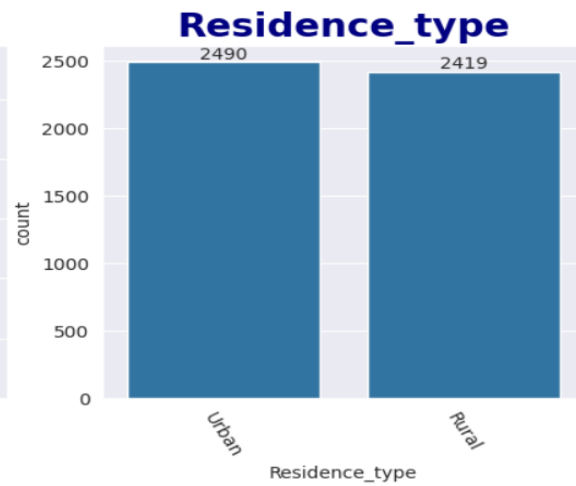
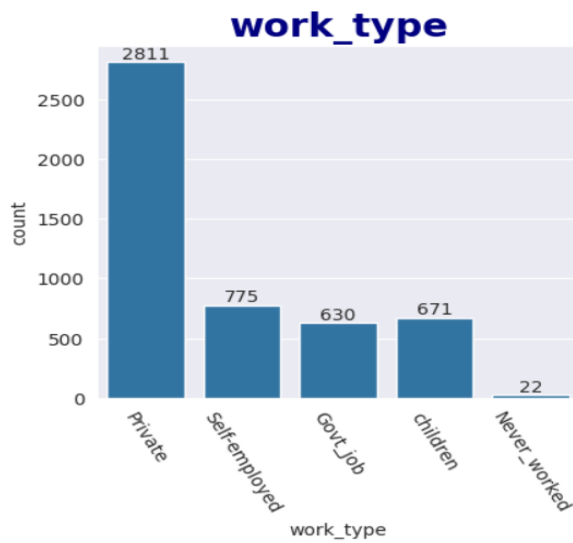
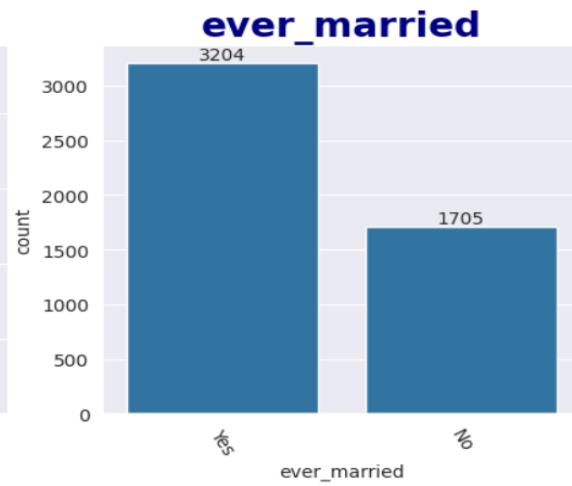
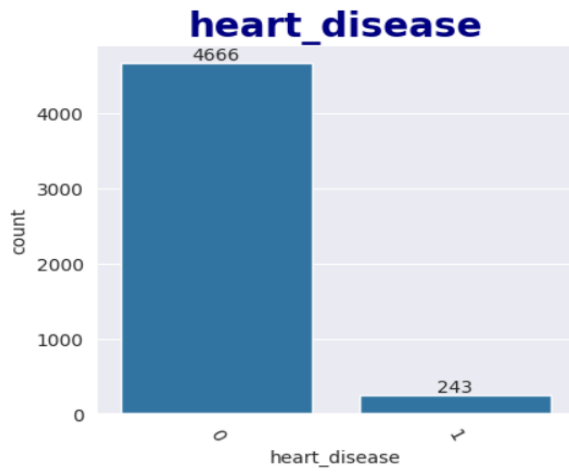
i=0
while i<8 :

    # Left AX
    fig = plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.title(categorical_columns[i], size=20, weight='bold', color='navy')
    ax = sns.countplot(x=categorical_columns[i], data=data)
    ax.bar_label(ax.containers[0])
    ax.tick_params(axis='x', rotation=300)
    i+=1

    # Right AX
    plt.subplot(1, 2, 2)
    plt.title(categorical_columns[i], size=20, weight='bold', color='navy')
    ax = sns.countplot(x=categorical_columns[i], data=data)
    ax.bar_label(ax.containers[0])
    ax.tick_params(axis='x', rotation=300)
    i+=1
plt.show()

```

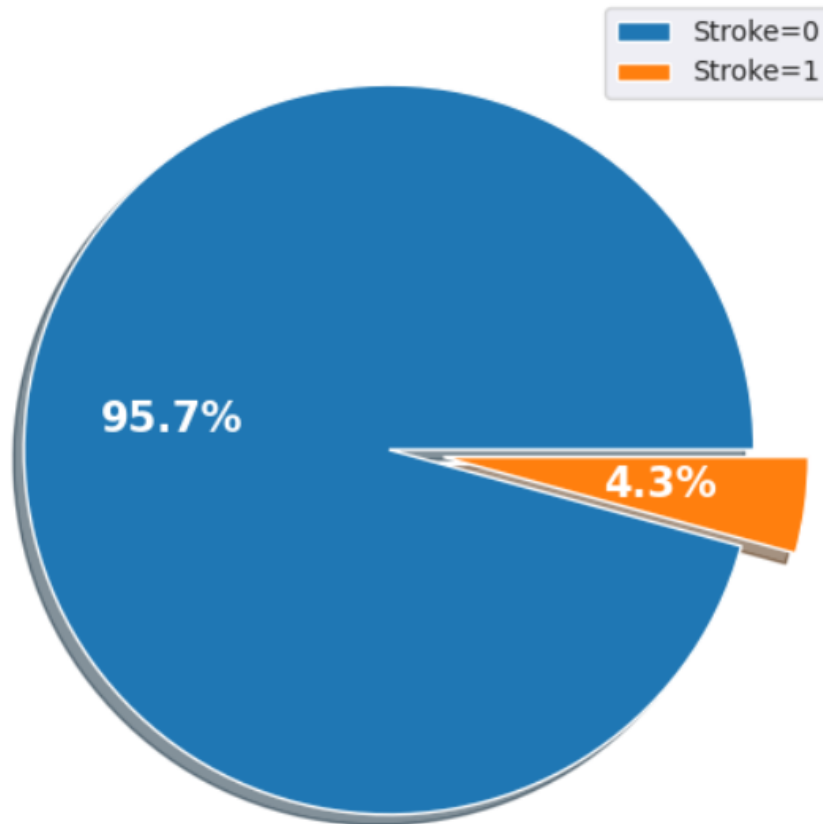




```
x = data['stroke'].value_counts()

explode = [0, 0.15]
labels = ['Stroke=0', 'Stroke=1']
fig, ax = plt.subplots(figsize=(6, 6), subplot_kw=dict(aspect="equal"))

plt.pie(x, explode=explode, shadow=True, autopct='%1.1f%%', labels=labels, textprops=dict(color="w", weight='bold', size=15))
plt.legend()
plt.show()
```



EXPLANATION

About 96% of samples have not had a stroke, and 4% have had a stroke

- This indicates that strokes are relatively infrequent in the dataset, with a vast majority of samples not experiencing strokes.
- It suggests a class imbalance issue that might need to be addressed during model training, such as using techniques like oversampling or undersampling.

The distribution of samples is a Normal distribution

- A normal distribution implies that the data is symmetrically distributed around the mean, with the majority of samples clustered around the mean and fewer samples towards the tails.
- It suggests that certain statistical assumptions, like normality, may be valid, which can facilitate certain analyses or model assumptions.

Characteristics of individuals who have had a stroke

- Age: Falls in the range of 40 to 85 years.
- BMI (Body Mass Index): Falls in the range of 20 to 40.
- Glucose level: Falls in the range of 50 to 130.
- This information provides insights into the demographic and health profile of individuals who have experienced strokes.

About 60% of samples are female

- Indicates that females are slightly overrepresented in the dataset compared to males.
- Gender could be an important feature in predicting stroke risk and might need to be considered during model training.

About 91% of samples don't have any hypertension

- Hypertension is relatively uncommon in the dataset, with the vast majority of samples not exhibiting this condition.
- It's essential to consider hypertension as a potential risk factor for strokes and its impact on model predictions.

About 95% of samples don't have any heart disease

- Similar to hypertension, heart disease is also relatively infrequent in the dataset.
- Understanding the prevalence of comorbidities like heart disease can aid in assessing stroke risk factors comprehensively.

About 34% of samples never get married

- Provides insights into the marital status distribution of the dataset.

- Marital status might be associated with various lifestyle factors and social support, which can influence stroke risk.

Most of the samples worked in private

- Indicates the dominant employment type among individuals in the dataset.
- Occupational factors might influence lifestyle choices and stress levels, which can impact health outcomes like strokes.

Missing information in the smoking field for 1483 samples

- Indicates a significant portion of missing data in the smoking attribute.
- Missing data handling strategies like imputation or treating missingness as a separate category might be necessary to retain information from these samples without biasing the analysis.

UNIQUE VALUES

```
columns_temp = ['gender', 'ever_married', 'work_type', 'smoking_status', 'Residence_type']

for col in columns_temp :
    print('column :', col)
    for index, unique in enumerate(data[col].unique()) :
        print(unique, ': ', index)
    print('_'*45)
```

```
column : gender
Male : 0
Female : 1
Other : 2
```

```
column : ever_married
Yes : 0
No : 1
```

```
column : work_type
Private : 0
Self-employed : 1
Govt_job : 2
children : 3
Never_worked : 4
```

```
column : smoking_status
formerly smoked : 0
never smoked : 1
smokes : 2
Unknown : 3
```

```
column : Residence_type
Urban : 0
Rural : 1
```

```

# gender
data_2 = data.replace(
    {'gender' : {'Male' : 0, 'Female' : 1, 'Other' : 2}}
)

# ever_married
data_2 = data_2.replace(
    {'ever_married' : {'Yes' : 0, 'No' : 1}}
)

# work_type
data_2 = data_2.replace(
    {'work_type' : {'Private' : 0, 'Self-employed' : 1, 'Govt_job' : 2, 'children' : 3, 'Never_worked' : 4}}
)

# smoking_status
data_2 = data_2.replace(
    {'smoking_status' : {'formerly smoked' : 0, 'never smoked' : 1, 'smokes' : 2, 'Unknown' : 3}}
)

# Residence_type
data_2 = data_2.replace(
    {'Residence_type' : {'Urban' : 0, 'Rural' : 1}}
)

```

```
data_2.head()
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	0	67.0	0	1	0	0	0	228.69	36.6	0	1
2	0	80.0	0	1	0	0	1	105.92	32.5	1	1
3	1	49.0	0	0	0	0	0	171.23	34.4	2	1
4	1	79.0	1	0	0	1	1	174.12	24.0	1	1
5	0	81.0	0	0	0	0	0	186.21	29.0	0	1

EXPLANATION

The `.unique()` command is a function in pandas that is used to find the unique elements in a Series or DataFrame. When applied to a DataFrame column, it returns an array containing all the unique values present in that column

NORMALIZATION

```
X_temp = data_2.drop(columns='stroke')
y = data_2.stroke
```

```
scaler = MinMaxScaler().fit_transform(X_temp)
X = pd.DataFrame(scaler, columns=X_temp.columns)
X.describe()
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status
count	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000	4909.000000
mean	0.295274	0.522282	0.091872	0.049501	0.347321	0.210634	0.492768	0.231674	0.212981	0.527942
std	0.246098	0.275331	0.288875	0.216934	0.476167	0.281014	0.499999	0.205080	0.089966	0.363382
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.304199	0.000000	0.000000	0.000000	0.000000	0.000000	0.101330	0.151203	0.333333
50%	0.500000	0.536133	0.000000	0.000000	0.000000	0.000000	0.000000	0.168775	0.203895	0.333333
75%	0.500000	0.731445	0.000000	0.000000	1.000000	0.500000	1.000000	0.269827	0.261168	1.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

EXPLANATION

The data by separating the features (X_temp) from the target variable (y), scales the features using MinMaxScaler to ensure they are within the same range, and creates a new DataFrame X containing the scaled features. Finally, it provides descriptive statistics for the scaled features to understand their distribution and range. This preprocessing step is essential for ensuring that the features are on a similar scale, which can improve the performance of certain machine learning algorithms

INITIALIZATION

```
# define a function to plotting Confusion matrix
def plot_confusion_matrix(y_test, y_prediction):
    cm = metrics.confusion_matrix(y_test, y_prediction)
    ax = plt.subplot()
    ax = sns.heatmap(cm, annot=True, fmt='', cmap="Greens")
    ax.set_xlabel('Prediced labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix')
    ax.xaxis.set_ticklabels(['Dont Had Stroke', 'Had Stroke'])
    ax.yaxis.set_ticklabels(['Dont Had Stroke', 'Had Stroke'])
    plt.show()
```

```
# Splite X, y to train & test dataset.
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)
```

EXPLANATION

Function to Plot Confusion Matrix

- The provided code defines a Python function named `plot_confusion_matrix` that takes two arguments: `y_test` (true labels) and `y_prediction` (predicted labels).
- Inside the function, it calculates the confusion matrix using `metrics.confusion_matrix` from `sklearn`.
- Then, it creates a heatmap visualization of the confusion matrix using `seaborn's` `heatmap` function.
- The heatmap is annotated with the values of the confusion matrix.
- Axes labels and title are set to provide context to the visualization.
- Finally, it displays the plot using `plt.show()` from `matplotlib`.

Splitting X and y into Train and Test Sets

- The code splits the feature matrix `X` and the target vector `y` into training and testing datasets using `train_test_split` from `sklearn`.
- It assigns 75% of the data to the training set (`X_train` and `y_train`) and 25% to the test set (`X_test` and `y_test`).
- The `random_state` parameter is set to 0 to ensure reproducibility of the split.
- `test_size=0.25` specifies that 25% of the data should be used for testing, leaving the remaining 75% for training.

LOGISTIC REGRESSION

```
# a dictionary to define parameters to test in algorithm
parameters = {
    'C' : [0.001, 0.01, 0.1, 1.0, 10, 100, 1000],
    'class_weight' : ['balanced'],
    'solver' : ['lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga']
}

lr = LogisticRegression()
lr_cv = GridSearchCV(estimator=lr, param_grid=parameters, cv=10).fit(X_train, y_train)

print('Tuned hyper parameters : ', lr_cv.best_params_)
print('accuracy : ', lr_cv.best_score_)
```

```
Tuned hyper parameters : {'C': 100, 'class_weight': 'balanced', 'solver': 'sag'}
accuracy : 0.7530679274184046
```

```
# Calculate time before run algorithm
t1 = datetime.now()
# Model
lr = LogisticRegression(**lr_cv.best_params_).fit(X_train, y_train)
# Calculate time after run algorithm
t2 = datetime.now()
```

```
y_pred_lr = lr.predict(X_test)

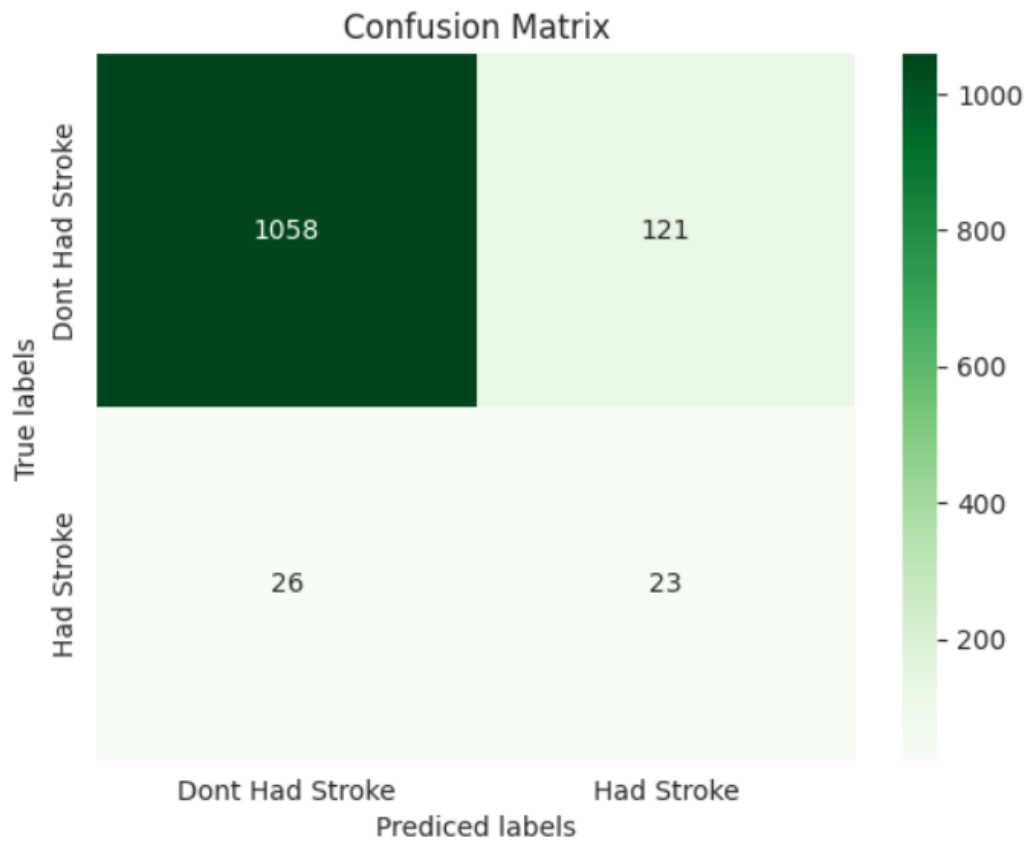
lr_score = round(lr.score(X_test, y_test), 3)
print('LogisticRegression score : ', lr_score)
```

```
LogisticRegression score : 0.88
```

```
delta = t2-t1
delta_lr = round(delta.total_seconds(), 3)
print('LogisticRegression takes : ', delta_lr, 'Seconds')
```

LogisticRegression takes : 0.086 Seconds

```
plot_confusion_matrix(y_test, y_pred_lr)
```



```
cr = metrics.classification_report(y_test, y_pred_lr)
print(cr)
```

	precision	recall	f1-score	support
0	0.98	0.90	0.94	1179
1	0.16	0.47	0.24	49
accuracy			0.88	1228
macro avg	0.57	0.68	0.59	1228
weighted avg	0.94	0.88	0.91	1228

CONCLUSION

The Logistic Regression model achieved an accuracy score of approximately 59.7% on the test dataset. This accuracy score indicates that the model correctly predicted the presence or absence of strokes for nearly 60% of the samples in the test set.